AIORI HACKATHON 3 · PS-004

A3-PS004-TC399

Authenticated Gateway-Assisted DNSSEC for Resource-Constrained IoT

Theme: IoT and Cloud Computing



TATA^2

Anvesha · Heritage Institute of Technology, Kolkata
anvesha.aeie28@heritageit.edu.in

Vardan Singh · Heritage Institute of Technology, Kolkata
vardan.singh.aeie28@heritageit.edu.in

14.10.2026

Abstract & Problem Description

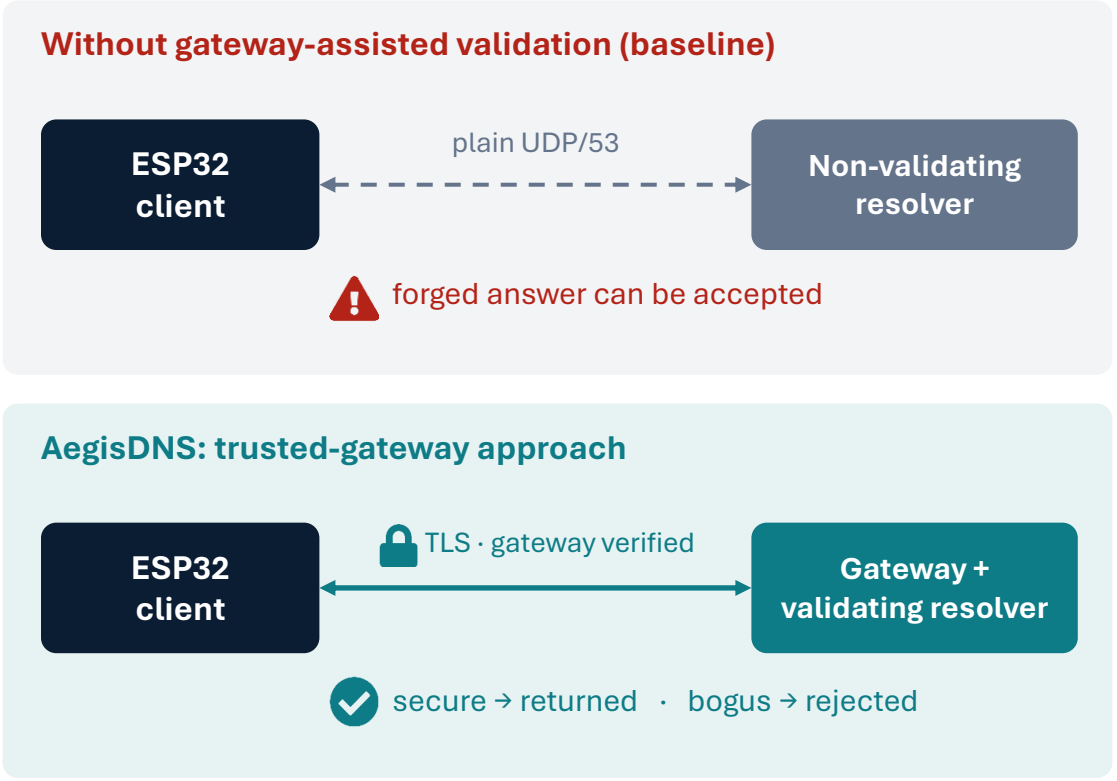
Small devices need trustworthy DNS answers but cannot easily validate DNSSEC themselves

Abstract

AegisDNS lets constrained IoT clients (e.g., ESP32) obtain DNSSEC-validated answers without validating locally. The client first authenticates a trusted gateway and opens an encrypted channel. The gateway forwards each query to its DNSSEC-validating resolver, applies an explicit policy to the validation result, returns acceptable answers and rejects bogus ones. Latency, CPU and memory are benchmarked against a plain-DNS baseline.

Key problems

- IoT devices rely on DNS to reach cloud, API and update services
- Attackers can forge or alter DNS responses in transit
- DNSSEC adds integrity + origin authentication via a chain of trust [1]
- Full validation costs code, crypto and trust-anchor upkeep on MCUs [7]
- A secure channel alone does not prove the data was DNSSEC-validated
- So the gateway must be authenticated and must reject bogus results



Problem statement How can a constrained IoT client obtain DNSSEC-validated answers without validating locally (trusting only a gateway it has authenticated) while bogus, insecure and failed lookups are never passed off as secure?

Background

Two different protections: DNSSEC protects the data, TLS/DTLS protects the channel

Concept	What it does	What it does NOT do	Role in AegisDNS
DNS resolution	Maps names (api.example.org) to addresses via resolvers	Prove that an answer is authentic	Base service the client needs
DNSSEC validation	Checks RRSIG / DNSKEY / DS up to a trust anchor → secure, insecure, bogus or indeterminate [1][2][3]	Encrypt DNS, or say a site is harmless	Done by the gateway's validating resolver
Gateway-assisted processing	A trusted nearby node does heavy work for many devices	Remove the need to trust that node	Offloads validation from the MCU
TLS/DTLS + gateway authentication	Encrypts the client–gateway link; certificate + name check proves gateway identity [4][5][6]	Validate DNS data	Secure, authenticated client–gateway channel
IoT resource constraints	Small RAM, flash, CPU and energy budgets (RFC 7228 device classes) [7]	—	Why validation is offloaded

DNSSEC validates DNS data : it does not encrypt DNS traffic [1]

TLS/DTLS protects the client–gateway channel [4][6]

The client authenticates the gateway (certificate + name) [5]

The gateway's resolver performs DNSSEC validation [3]

Literature & RFC Review

Each standard maps to a concrete design decision

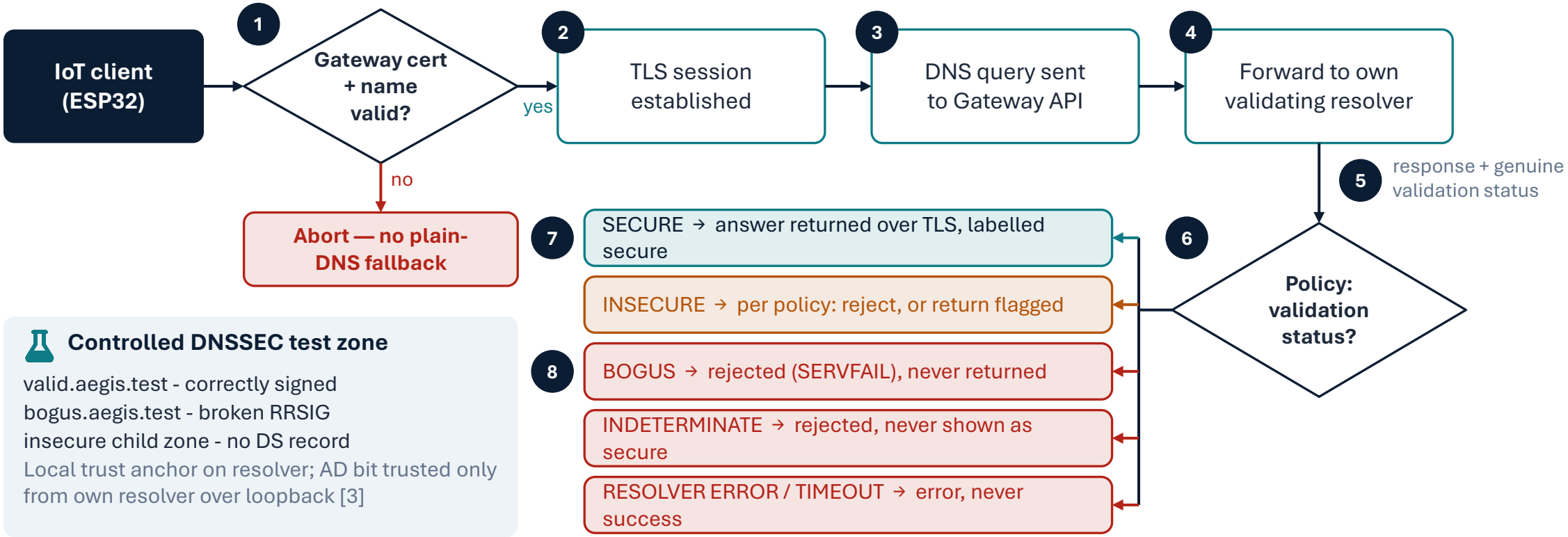
Standard	Core purpose	Relevance to AegisDNS	Design decision it informs
RFC 4033 [1] DNSSEC Introduction & Requirements	Defines DNSSEC services and outcomes: secure, insecure, bogus, indeterminate	Defines what “validated” means; DNSSEC gives no confidentiality	Policy maps each of the four outcomes to an explicit action
RFC 4034 [2] Resource Records for DNSSEC	Specifies DNSKEY, RRSIG, NSEC and DS records	Records our test zone must publish and the resolver checks	Signed test zone; deliberately broken RRSIG for bogus cases
RFC 4035 [3] Protocol Modifications for DNSSEC	Resolver behaviour; DO, AD, CD bits; stub trust (§4.9)	Stub must not rely on remote validation except via a trusted resolver over a secure channel	Gateway trusts AD only from its own resolver over loopback; client CD bit never forwarded
RFC 7858 [4] DNS over TLS	Carries DNS over TLS (port 853) between stub and resolver	Pattern for encrypting client–gateway DNS traffic	Client–gateway transport is DNS-over-TLS; DTLS evaluated as a variant
RFC 9525 [5] Service Identity in TLS	How a client verifies a server's identity (DNS-ID) in its certificate	Client must prove it reached the real gateway	Check cert chain + DNS-ID; any failure aborts, no plain-DNS fallback

Also used: RFC 8310 [6] - DoT/DTLS usage profiles (strict profile requires server authentication) · RFC 7228 [7] - constrained-node device classes

Our contribution is an engineering one: implement and evaluate an authenticated, gateway-assisted DNSSEC workflow for constrained IoT clients, with an explicit outcome policy and a repeatable benchmark. Validation at a trusted resolver is established practice; we do not claim novelty.

Proposed Solution

The client trusts one authenticated gateway; DNSSEC validation happens only in the gateway's resolver



Controlled DNSSEC test zone

valid.aegis.test - correctly signed
bogus.aegis.test - broken RRSIG
insecure child zone - no DS record
Local trust anchor on resolver; AD bit trusted only from own resolver over loopback [3]

9 Every path is logged: query, outcome, policy action, latency, gateway CPU / memory → structured event + benchmark log

Solution Optimization

Intended design differences against a clearly defined baseline (not measured results)

Baseline: ESP32 → plain UDP DNS → non-validating resolver; no gateway authentication

Aspect	Baseline	AegisDNS (intended)
DNS validation	None enforced	Validation by the gateway's configured resolver
Client–gateway security	Plain UDP/53, unauthenticated	Authenticated, encrypted TLS channel
Client validation burden	None (but no DNSSEC protection either)	DNSSEC processing offloaded to gateway
Bogus DNS handling	No safeguard; forged data passes	Explicit rejection from genuine validation status
Monitoring	Basic or absent	Structured security + performance logs
Evaluation	Limited metrics	Repeatable latency, CPU, memory and outcome measurements

Trade-offs we will measure, not hide

Gateway dependency
Single point of failure; fail closed + health checks

Extra latency
One more hop plus gateway processing

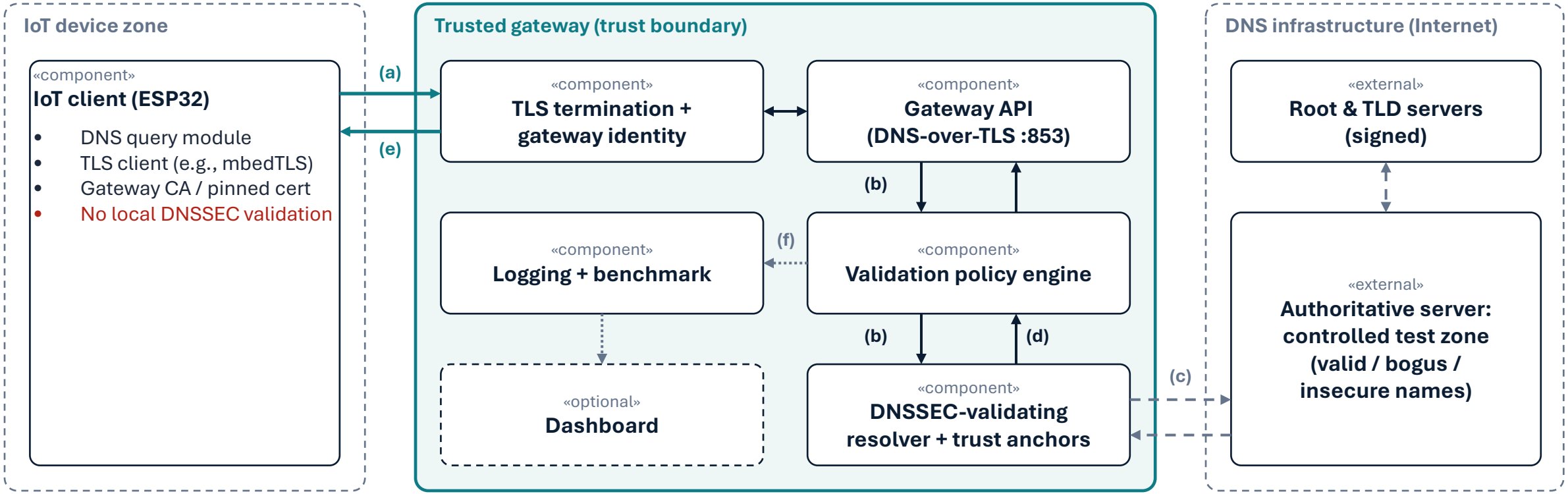
TLS handshake cost
Reduced by session reuse / persistent connection

Gateway resources
CPU and memory measured under load

Trust in gateway + resolver
A compromised gateway defeats the scheme: harden, pin

Solution Architecture & UML Diagram

Component view: what runs where, and which links cross a trust boundary



(a) client → gateway: DNS query over TLS

(b) query down / answer up, gateway-internal

(c) resolver ↔ DNS: recursive queries with DO=1

(d) resolver → policy: answer + validation status

(e) gateway → client: answer or rejection over TLS

(f) events + metrics → log

TLS-protected link

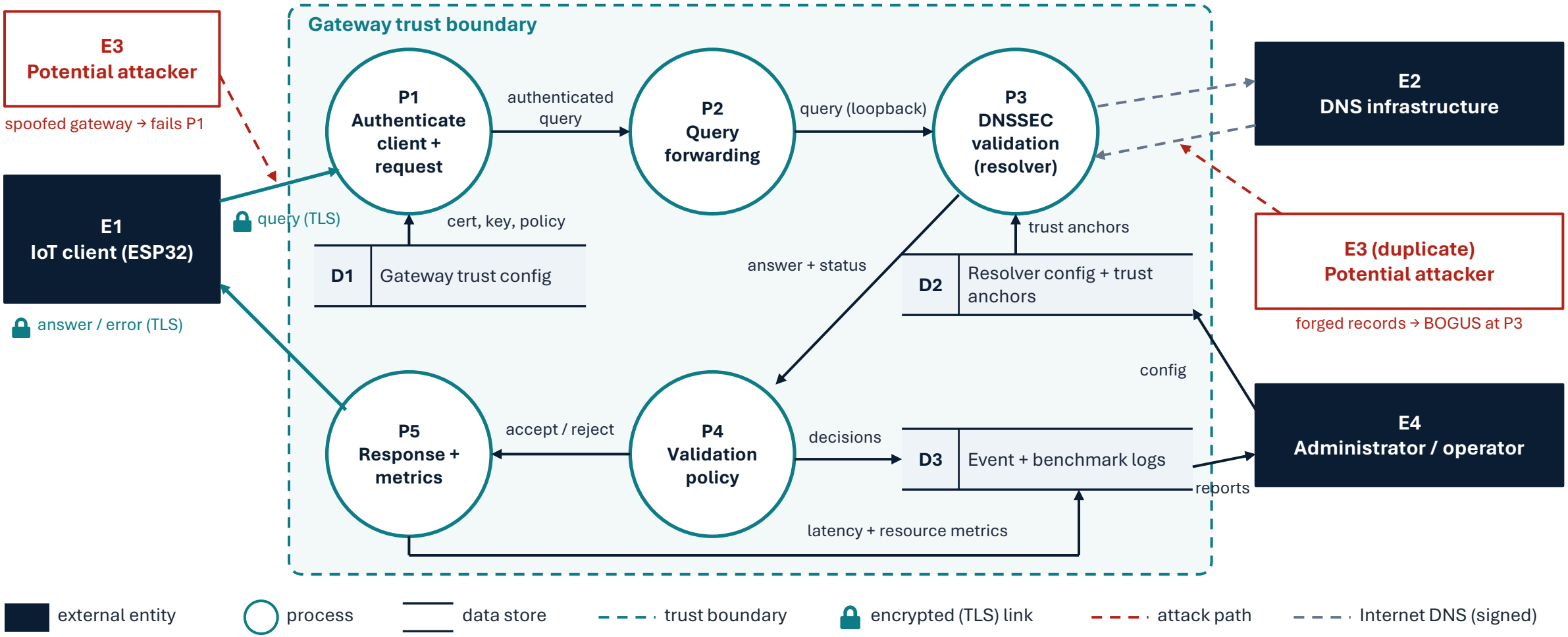
gateway-internal (loopback)

Internet DNS: signed, not encrypted

metrics / logging

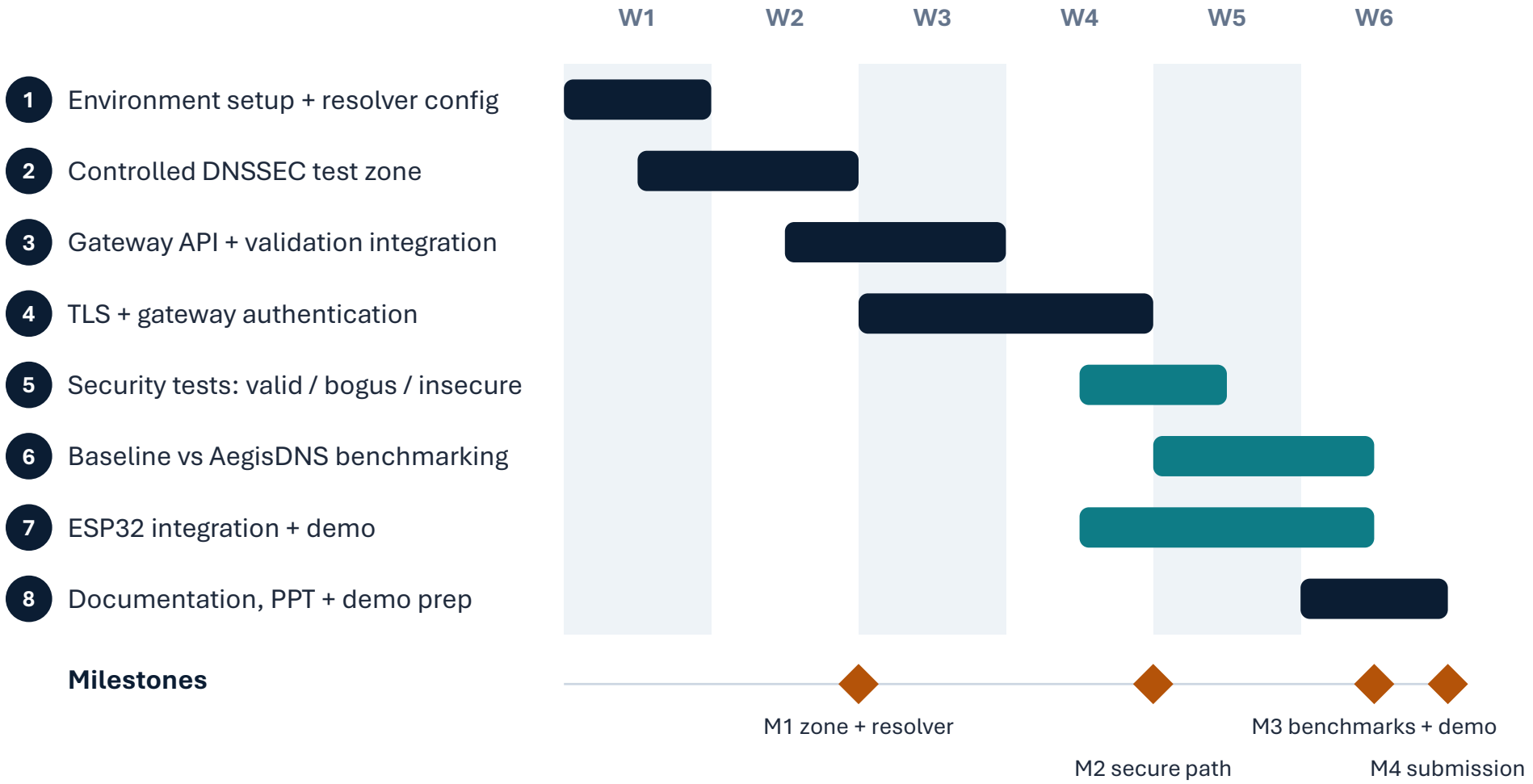
Data Flow Diagram

Where data moves, where it is stored, and where an attacker can reach it



Deployment Timeline

Proposed timeline in relative weeks - all phases are planned, none reported as complete



Measurement plan

- Latency per query (median, p95) by outcome
- Gateway CPU % and resident memory
- ESP32 free heap + TLS handshake time
- Correctness: valid accepted, bogus rejected, insecure per policy, errors never success
- Same network, repeated runs, baseline vs AegisDNS

Bibliography

All entries checked on rfc-editor.org; numbers match the in-slide citations

[1] R. Arends, R. Austein, M. Larson, D. Massey, S. Rose. “DNS Security Introduction and Requirements.” RFC 4033, March 2005.
https://www.rfc-editor.org/rfc/rfc4033.html

[2] R. Arends, R. Austein, M. Larson, D. Massey, S. Rose. “Resource Records for the DNS Security Extensions.” RFC 4034, March 2005.
https://www.rfc-editor.org/rfc/rfc4034.html

[3] R. Arends, R. Austein, M. Larson, D. Massey, S. Rose. “Protocol Modifications for the DNS Security Extensions.” RFC 4035, March 2005.
https://www.rfc-editor.org/rfc/rfc4035.html

[4] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, P. Hoffman. “Specification for DNS over Transport Layer Security (TLS).” RFC 7858, May 2016.
https://www.rfc-editor.org/rfc/rfc7858.html

[5] P. Saint-Andre, R. Salz. “Service Identity in TLS.” RFC 9525, November 2023.
https://www.rfc-editor.org/rfc/rfc9525.html

[6] S. Dickinson, D. Gillmor, T. Reddy. “Usage Profiles for DNS over TLS and DNS over DTLS.” RFC 8310, March 2018.
https://www.rfc-editor.org/rfc/rfc8310.html

[7] C. Bormann, M. Ersue, A. Keranen. “Terminology for Constrained-Node Networks.” RFC 7228, May 2014.
https://www.rfc-editor.org/rfc/rfc7228.html

Thank You for Your Attention!

A3-PS004-TC399 · PS-004 · IoT and Cloud Computing

